SIMATS ENGINEERING
ASSESSMENT TOOL 1
COURSE NAME: Artificial Intelligence
COURSE CODE: CSA17

Code of Conduct:

I, **B.Vishnu** / **192512410** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

ANNEXURE A
DESIGN TASK - SOLUTIONS

1. Hybrid AI Recommendation Engine Design

System Architecture & Data Flow: The architecture begins with a Data Ingestion Layer capturing real-time clickstreams and static user profiles. This feeds into a distributed PySpark processing pipeline. The pipeline forks into two streams: a Content-Based filtering engine (analyzing item metadata) and a Collaborative Filtering engine (computing user-item similarity matrices). The outputs are fused by an ensemble model to generate a final ranked recommendation list.

PySpark & RDD Operations: Handling millions of user-item interactions requires distributed computing. RDDs (Resilient Distributed Datasets) partition the massive interaction logs across cluster nodes. Operations like `map()` are used to parse raw logs into (user_id, item_id, rating) tuples. `groupByKey()` aggregates all historical interactions per user, and `join()` combines the user's interaction history with the item metadata RDD. This parallelization ensures scalability and fault tolerance.

Intelligent Agent Integration: Intelligent agents act as the real-time adaptation layer. A "User Context Agent" monitors live session behavior. If a user historically watches Sci-Fi (captured in the batch RDD process) but suddenly clicks on three Comedy movies, the agent dynamically adjusts the hybrid engine's weighting to heavily favor the content-based Comedy stream for the remainder of the session, ensuring highly responsive personalization.

2. Smart Disaster Detection System

System Architecture & Data Flow: The system requires a highly scalable lambda architecture. Real-time streams from IoT seismic sensors, water level monitors, and social media APIs (Twitter) are ingested via Kafka. Satellite imagery is processed in batches. These diverse formats are fused into a unified PySpark processing pipeline to continuously evaluate regional risk scores.

PySpark & RDD Operations: PySpark RDDs are critical for data fusion. Structured sensor data is mapped to (region_id, threshold_variance) using `map()`. Unstructured social media text undergoes distributed NLP processing using `flatMap()` to tokenize words and extract disaster-related keywords. Finally, a `reduceByKey()` operation aggregates the multi-modal risk scores by geographic region, efficiently computing localized threat levels across a distributed cluster.

Intelligent Agent Integration: A Multi-Agent System (MAS) handles the response. A "Data Fusion Agent" monitors the RDD output. If an anomaly is detected, it alerts the "Verification Agent," which cross-references satellite imagery with social media sentiment. Upon confirmation, a "Coordinator Agent" autonomously dispatches early warnings to local authorities and triggers automated SMS alerts to the affected populace, operating entirely without human bottleneck delays.

3. AI-Based Fraud Detection Framework

System Architecture & Data Flow: The transaction gateway routes all global payment requests into a Spark Streaming environment. The data flows through a data enrichment layer (fetching historical user behavior) into the distributed anomaly detection model (e.g., Isolation Forest), which outputs a fraud probability score. Flagged transactions are then routed to the resolution agents.

PySpark & RDD Operations: Financial datasets are notoriously massive. RDDs allow in-memory computation, which is vital for sub-second fraud detection. We utilize `mapPartitions()` to calculate transaction velocity (transactions per minute) in parallel across nodes. RDD `filter()` operations rapidly drop normal transactions, while suspicious patterns are grouped using `reduceByKey()` to analyze geographic dispersion (e.g., a card used in New York and London within 5 minutes).

Intelligent Agent Integration: Agents manage the intervention workflow. An "Alert Agent" intercepts high-risk scores. Instead of outright blocking a borderline transaction, a "Mitigation Agent" can autonomously trigger a step-up authentication (like a 2FA SMS code). The outcomes (Success/Fail) are captured by a "Learning Agent," which continuously feeds this labeled data back into the PySpark training pipeline to minimize future false positives.

4. Autonomous Supply Chain Optimization System

System Architecture & Data Flow: The architecture connects global ERP systems, warehouse databases, and transit GPS feeds into a centralized PySpark data lake. The processing pipeline computes global inventory states, demand forecasts, and logistical delays, pushing the analyzed state to a decentralized network of intelligent agents.

PySpark & RDD Operations: RDDs handle the massive scale of SKU-level tracking. Daily sales data is transformed using `map()` and aggregated using `reduceByKey()` to calculate aggregate demand per region. `join()` operations merge the demand RDD with the current transit RDD to identify projected stockouts. The fault-tolerant nature of RDDs ensures that if a server node analyzing the European market fails, the computation is automatically re-run on another node.

Intelligent Agent Integration: The system employs decentralized decision-making. A "Warehouse Agent" in Berlin detects an impending shortage of a component (flagged by the PySpark pipeline). It autonomously negotiates with a "Logistics Agent" overseeing transit routes to dynamically reroute a shipping container currently headed to Paris, optimizing for cost and urgency without requiring central human logistics management.

5. Personalized Learning Analytics Platform

System Architecture & Data Flow: Learning Management Systems (LMS) stream student interaction data (video pauses, quiz attempts, forum posts) into a PySpark processing cluster. The cluster extracts behavioral features and updates student knowledge vectors. These vectors are queried by pedagogical agents to curate personalized dashboard experiences.

PySpark & RDD Operations: Scaling to thousands of learners requires mapping complex behavioral data. We use `flatMap()` to unpack JSON clickstreams. A `map()` function calculates the time spent on specific concepts. We then use `aggregateByKey()` to compute average mastery scores per topic for each student. Distributed processing allows the system to recalculate the learning curves of a massive student body overnight or in near-real-time.

Intelligent Agent Integration: The platform utilizes "Pedagogical Agents." If the RDD analytics reveal a student has failed two consecutive quizzes on "Thermodynamics" and frequently pauses the corresponding lecture video, the agent intervenes. It autonomously alters the student's learning path, injecting foundational prerequisite materials, visual simulations, or easier practice problems into their feed to prevent frustration, ensuring a highly adaptive and personalized educational experience.